第十四届“恩智浦”杯全国大学生
智能汽车竞赛
室外光电创意组

技 术 报 告

参赛学校：清华大学

队伍名称：紫荆之巅

参赛队员：胡若飞

占奥铭

辜俊皓

杨雨杰

游子权

带队教师：朱小亮

关于技术报告和研究论文使用授权的说明

本人完全了解第十四届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：胡若飞 占奥然 韦俊锴 杨雨辰 授权

带队教师签名：朱华勇

日　　期：2019.3.31

目录

第一章　引言 1
第二章　小车制作部分 2
第三章　智能车硬件电路设计 3
3.1 PCB 布线设计 3
3.2 供电模块电路 3
3.2.1 单片机供电电路 3
3.2.2 雷达供电电路 4
3.3 重要接口 5
3.3.1 USB 转串口电路（电脑——单片机） 5
3.4 小结 6
第四章 ROS 软件设计 7
4.1 ROS 软件系统概述 7
4.2 地图生成 7
4.3 定位 8
4.4 路径修正与避障 9
4.5 单片机驱动模块 9
4.6 小车运行文件和控制参数介绍 9
4.6.1 运行文件 9
4.6.2 控制参数 10
4.7 udev 端口配对 11
4.8 键盘控制 12
第五章　单片机程序设计 13
5.1 基本原理 13
5.2 算法设计 13
5.2.1 串口通讯 13
5.2.2 PWM 输出 13
5.2.3 编解码与基本控制 13
第六章　结语 15

附录：程序源代码.....i

第一章　引言

本组为第十四届全国大学智能汽车室外光电竞速创意组。本次比赛旨在人工智能技术飞速发展的当下，通过场景化的比赛复现基于无人驾驶的智能车在实际领域中的应用。锻炼学生们在特定场景，尤其是在无人的环境中，研究并实现定位导航、计算机视觉、雷达、人工智能、自动控制和电机控制等当下被广泛应用的智能车控制技术。参加本次比赛，对于我们掌握机械电子、运动控制、传感器映红、机器学习、图像识别、SLAM 地图构建、自主导航等人工智能领域先进技术有很大帮助，能帮助我们走出课堂，从实践中感受 ROS 操作平台、激光雷达和 SLAM 等工业中广泛应用的技术，达到校园与业界接轨的效果。

本对目前已经完成了 PCB 电路设计与调试、单片机译码模块的控制与调试、ROS 操作系统的基本运行与初步调参，对小车的进一步优化和增强进行了前瞻性的调研。接下来将对本队的设计进行具体阐述。

第二章　小车制作部分

小车由车体、主控器电脑、路由器、激光雷达、姿态传感器、单片机、直流电机、舵机、电池等组成。激光雷达采集小车四周物体的距离，姿态传感器采集小车的加速度和角速度。单片机与主控电脑端通过 USB 转串口连接，单片机接收 ROS 系统下达的速度与角速度指令，输出 PWM 控制直流电机和舵机，进而控制小车的速度与方向。通过路由器组件局域网，可以使用 ssh 远程访问登陆主控电脑，方便远程控制与调试。

第三章　智能车硬件电路设计

硬件电路的主要功能有为各模块供电、实现各模块间的通讯与提供软件控制的载体，是决定智能小车性能的关键因素。下面从 PCB 布线、供电模块与重要接口三个方面阐述本队制作的硬件电路。

3.1 PCB 布线设计

PCB 设计本着缩小主板面积、简化布线电路与避免寄生电容电感的原则进行设计。在设计时我们也考虑了如智能小车其它模块的连接的便利性，对电路尤其是接口的拓扑结构进行了优化设计。设计的效果图如下图所示：

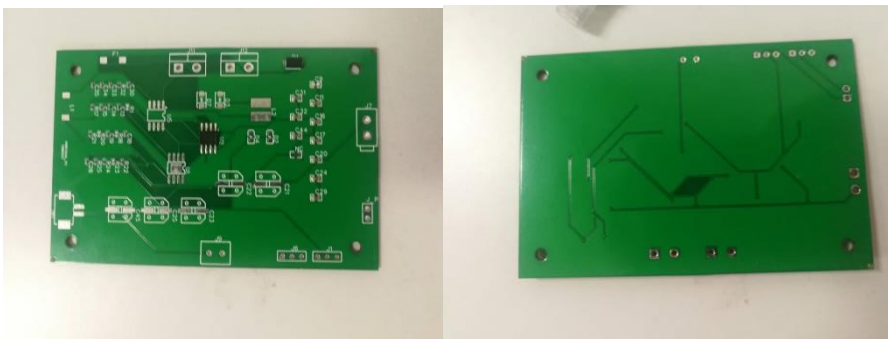


图 3.1　主控板设计效果

3.2 供电模块电路

3.2.1 单片机供电电路

单片机供电电路同时为单片机、舵机和路由器进行供电。将通过电源输入电路之后的电源接入到最大电流为 5A 的降压 DC-DC 芯片（TPS54231DR）上，再通过电阻分压，将输出电压稳定再 5V。参考数据手册，设计电路原理图如下：

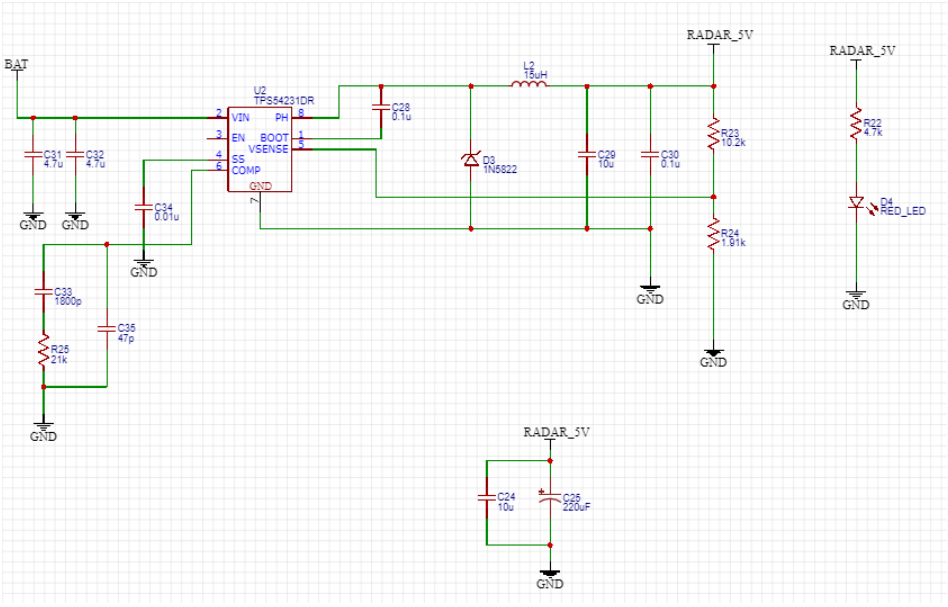


图 3.2　单片机供电电路原理图

3. 2. 2 雷达供电电路

雷达供电电路与单片机供电电路原理图近似，也是通过降压 DC-DC 芯片（TPS54560DDAR）与电阻分压将电源输入电路的输出稳定到 5V。参考数据手册，设计电路原理图如下：

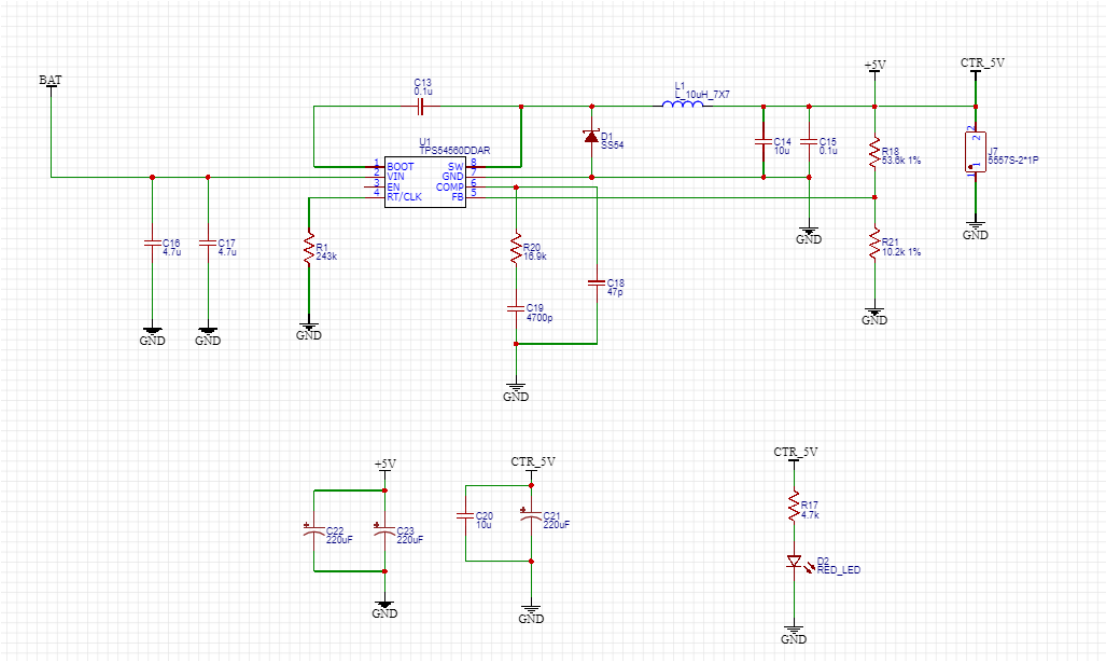


图 3.3 雷达供电电路原理图

3.3 重要接口

3.3.1 USB 转串口电路（电脑——单片机）

电路模块中，包含 7.4V 电池接口，11.1V 电池接口，雷达供电接口、单片机输入接口和舵机电调 PWM 输出接口。这部分电路设计的重点是根据智能小车哥模块，合理分配接口的位置和拓扑结构，以方便连接。电路原理图如下：

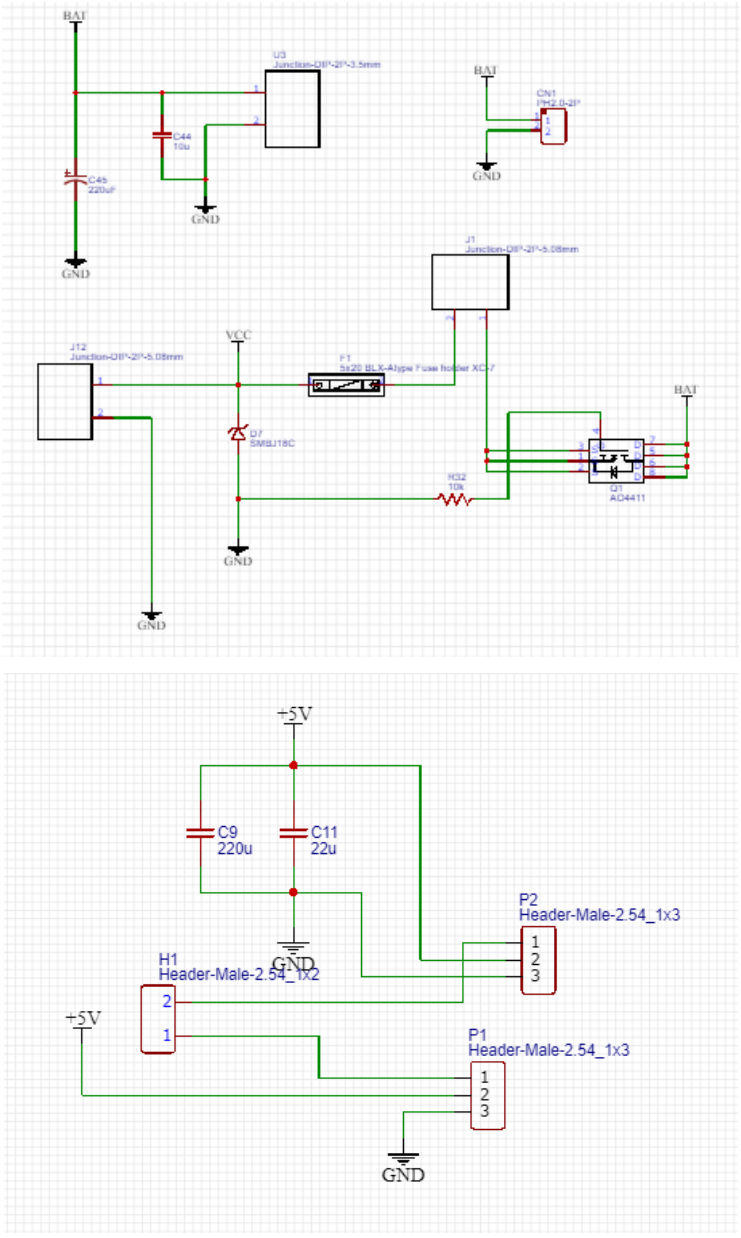


图 3.4　重要接口电路

3.4 小结

硬件电路的设计应当充分谨慎，并辅以适当的仿真。印刷好的电路应当进行充分调试，以保证小车的正常控制与运行。

第四章 ROS 软件设计

4.1 ROS 软件系统概述

软件系统的核心是 ROS（机器人操作系统），通过激光雷达和 IMU 姿态传感器获取小车周围的环境信息与小车的姿态信息，将采集到的数据进行处理，构建出整个环境的地图。比赛中在构建好的地图上，定位小车并规划出到终点的路径，同时避开随机的障碍。ROS 将控制信号传给单片机，单片机控制电机和舵机的转动。

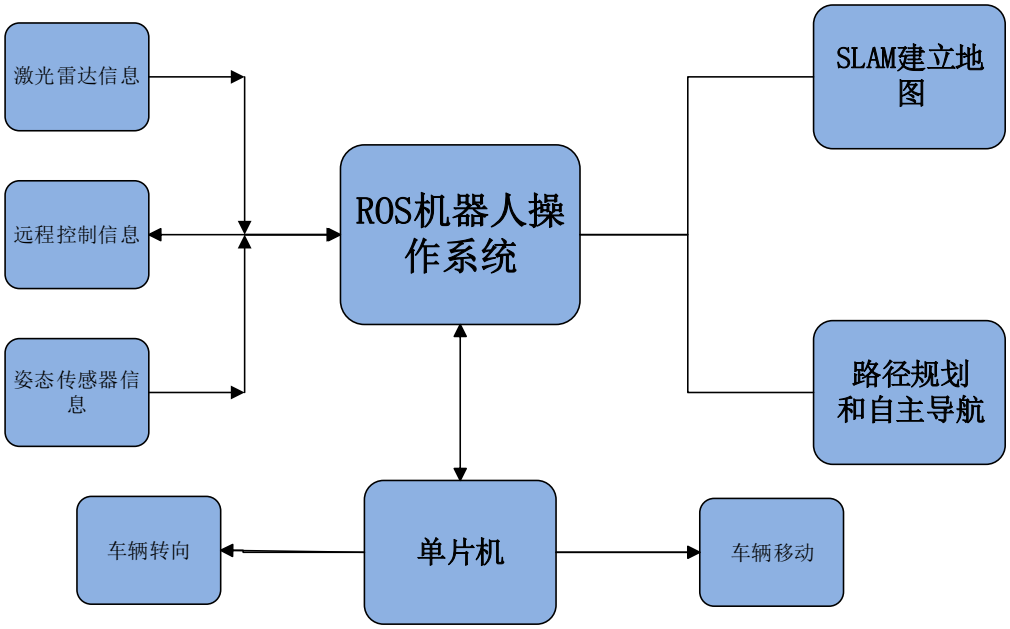


图 4.1　软件系统结构

4.2 地图生成

这一步在比赛中是会由比赛方提供，但在日常调试中还是需要自己构建地图以便后续操作，所以在此我们打算使用 ROS 中的 map_server 以及 gmapping 库来实现。而比赛官网上提供的键盘驱动.launch 文件以及 rviz 环境已经为我们的激光雷达提供了很好的数据处理，小车也能进行最基本的操控，接下来只需要

使用对应的 rosbag 指令记录下所有的存储数据后，调用 gmapping 库即可建立完整的地图。而关于地图的保存以及发布则可以利用 map_server 指令进行。

值得一提的是，以上过程中其实牵涉到的另一个内容就是 tf，tf 最基本的功能就是利用一种类似坐标树的结构，对多个参考系的相对位置进行实时跟踪，从而能够快速地在两个不同的参考系间进行坐标变换。通过这一功能，我们就可以将多个离散时间点的激光雷达接收到的数据进行拼接，从而得到全过程的完整的地图。

4.3 定位

定位是指小车确定自己在地图中的位置，通过 AMCL 算法来实现。

AMCL 是指自适应蒙特卡洛定位，用粒子过滤的方法实现在已知的二维地图上的定位。其工作原理是，将激光雷达检测到的周围环境与已知地图做比对，得到小车位置的概率分布（以粒子密度表示）。小车移动时，其位置的概率分布在地图上做与其相同的移动，同时激光雷达继续检测周围环境。将新检测到的周围环境与已知地图做比对，得到新的概率分布，将新的概率分布与原有概率分布叠加。不断重复这一过程，则小车附近的概率值将越来越大，概率分布会逐渐收敛到小车所在的位置。

ROS 中的 amcl 节点封装了 AMCL 算法，接收激光雷达、地图和 tf 变换信息的输入，输出小车位置的估计和概率分布。amcl 节点将基坐标系、里程计坐标系和全局坐标系连接起来，如下图所示。在接收到第一次激光扫描时，amcl 查找激光坐标系和基座坐标系之间的变换，并将其固定。通过估计基坐标系到全局坐标系的变换，amcl 可以消除里程计定位造成的累积误差。

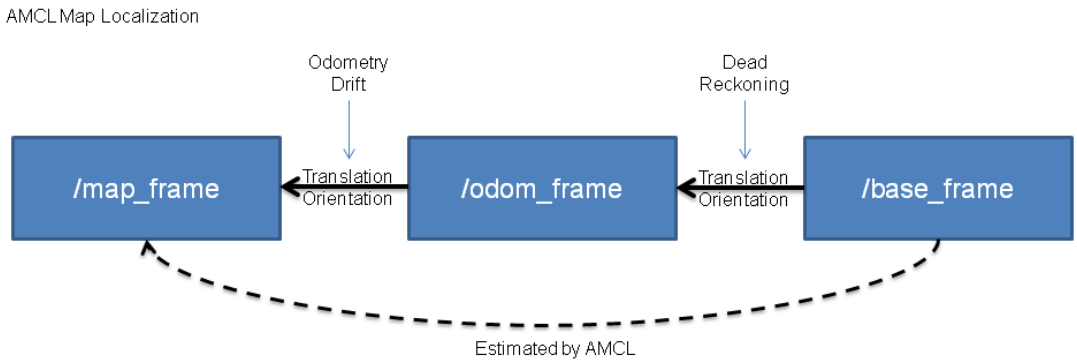


图 4.2　AMCL Map Localization

4.4 路径修正与避障

这一步主要用到的是 ROS 中的 navigation stack，例如 3.2 中提到的 map_server 也是这个库中的一部分。要利用该库，首先需要有一些基本的输入输出端口连接关系，例如传感器信息的接入，即激光雷达输入的信息；实时里程信息，这一点也是通过上述的 tf 库实现，还需要控制信息的输出，即前轮舵机以及后轮驱动的控制。除此之外，还有一些对环境和自身情况的先验知识，例如 3.2 中所构建出来的全局静态地图以及 3.3 中的实时定位。有了这些，navigation stack 工作的核心便可以开始维护两个地图，全局地图和局部地图，以及分别进行路径规划，基于 3.3 中提到的 amcl 算法，navigation stack 可以实时规划得到对应的最优路径，并通过输出端口连接到舵机以及驱动轮，从而完成路径修正与避障。

4.5 单片机驱动模块

电机和电调的控制信号通过小车电脑的 USB 接口传输至单片机，再由单片机进行译码控制电机电调。官方给的驱动模块在路径…/racecar/art_driver 下。我们对于 art_racecar.cpp 中调用的信息发送函数进行重写，在不改变接口和主要变量的情况下重新实现函数功能。

通过 ROS 自带的 serial 库即可实现 USB 端口的信息输出，需要设置好串口信息和波特率，就可以实现数据的发送。这样一来，通过回调函数对于获得角度和速度对应的控制信号，然后通过 serial 库的信息发送函数就可以将控制信号输出至单片机。

4.6 小车运行文件和控制参数介绍

小车的控制由一系列 launch 文件和控制参数决定，下面对其进行一一介绍。

4.6.1 运行文件

运行文件位于文件夹 launch/内，主要包括如下文件：

1）定位与导航文件（amcl_nav.launch），使用蒙特卡洛算法，用于运行小车的导航和定能。

2）小车运行文件（car.launch 和 Run_car.launch）car.launch 运行后进行 tf 变换并运行 car，Run_car.launch 运行后进行 tf 变换，并且运行 lidar，imu 和 car。

3）激光雷达运行文件（ls01g_lidar.launch）运行激光雷达。

4）地图建立与更新文件（Run_gmapping.launch）创建机器人地图，可以利用各种传感器融合、修正数据。将已经探知的环境特征、新的环境特征以及可能的噪声数据相关联，可以分析当前采集到的数据，修正之前数据所建立的地图，并利用新的数据增加地图信息，并过滤掉噪声数据。

5）rviz 运行文件（rviz.launch）运行 rviz，rviz 是 ros 自带的一个图形化工具，可以方便的对 ros 的程序进行图形化操作。

6）手柄控制文件（teleop_joy.launch）运行后可以手柄控制。

4.6.2 控制参数

1）导航和定位算法的参数（amcl.launch.xml）。

2）tf 变换的参数（art_car_tf.launch.xml）变换底盘，laser 和 imu 的坐标。

3）默认映射的参数（default_mapping.launch.xml）。

4）运行 laser 的参数（ls01g.launch.xml）。

5）运动测距参数（rf2o.launch.xml）通过连续范围内的扫描，对雷达的平面运动进行快速精确的估计。对于二维激光测距有着重要的价值。

6）激光雷达运行参数（rplidar.launch.xml）。

7）全局规划包参数（base_global_planner_params.yaml。）

8） dwa 包的参数（dwa_local_planner_params.yaml），该包使用 DWA（Dynamic Window Approach）算法实现了平面上移动机器人局部导航功能。 输入全局规划和代价地图后，局部规划器将会生成发送给移动底座的命令。dwa_local_planner 包实现了一个驱动底座移动的控制器，该控制器将路径规划器和机器人底座连在了一起。该规划器使用地图创建运动轨迹让机器人从起点到达目标点。移动过程中规划器会在机器人周围创建可以表示为珊格地图的评价函数。这里的控制器的主要任务就是利用评价函数确定发送给底座的速度（dx,dy,dtheta）。DWA 算法基本思路如下:首先，在机器人控制空间进行速度离

散采样(dx,dy,dtheta)。其次，对每一个采样速度执行前向模拟，看看使用该采样速度移动一小段段时间后会发生什么。然后，评价前向模拟中每个轨迹，评价准则如: 靠近障碍物，靠近目标，贴近全局路径和速度；丢弃非法轨迹（如哪些靠近障碍物的轨迹）。之后，挑出得分最高的轨迹并发送相应速度给移动底座。最后重复上面步骤。

9）进行扩展卡尔曼滤波的参数（ekf_params.yaml）其可以进行高效率的递归滤波。

10）生成全局代价地图的参数（global_costmap_params.yaml）。

11）生成本地代价地图的参数（local_costmap_params.yaml）。

12）路径规划的参数（move_base_params.yaml ）。

对于这些文件和参数的理解可以方便我们进行高效的调参，从而保证小车的高速稳定运行。

4.7 udev 端口配对

Linux 操作系统中，所有的外接设备使用前都需要经历挂载的过程。在本车中有主要有 3 个外接设备：小车底盘、雷达与 IMU。由于每一次挂载的端口号可能不同，需要利用 udev 设定规则，将挂载的设备制作固定的符号链结，使得 ROS 得以存取相关设备。以底盘驱动为例，我们可以将如下规则存成 car.rules 加入/etc/udev/rules.d 文件夹中。

```
KERNEL=="ttyUSB*", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", MODE:="666", SYMLINK+="car"
```

其中 ATTRS{idVendor} 与 ATTRS{idProduct}为与硬件相关的编号，用以绑定实际的硬件，而 SYMLINK 则指定符号链结后的名称，以上代码即将小车底盘连接至/dev/car 上。

4.8 键盘控制

利用键盘操控小车：用键盘控制小车的关键代码在 art_racecar/src/racecar_teleop.py 中。

```
moveBindings = {
        'i':(1,0),
        'o':(1,-1),
        'j':(0,1),
        'l':(0,-1),
        'u':(1,1),
        ',':(-1,0),
        '.':(-1,-1),
        'm':(-1,1),
        'I':(1,0),
        'O':(1,-1),
        'J':(0,1),
        'L':(0,-1),
        'U':(1,1),
        '<':(-1,0),
        '>':(-1,-1),
        'M':(-1,1),
        }
```

图 4.3　小车的键盘控制信号

moveBindings 为一个将键盘按键对应到一个参数元组（tuple）的字典（dict），参数元组（tuple）的第 1 个元素为速度方向，+1 为向前；0 为煞车，-1 为倒车（之后将会被引入到 speed_dir 变量中）；第 2 个元素为转向方向，+1 为左转，0 为直行，-1 为右转（之后将会于 turn_dir 变量中被调用）。最后向电机与舵机输出值由以下构成：

向电机输出 control_speed = speed_dir * (speed_start_value + speed_bias) + speed_mid，其中 speed_dir 为行进方向，speed_start_value + speed_bias 为速度增减的幅度，speed_mid 则为使电机不动的中间值。

向舵机输出 control_turn =turn_dir* (turn_start_value + turn_bias)+ turn_mid，其中 turn_dir 为转向方向，turn_start_value + speed_bias 为转向方向增减的幅度，speed_mid 则为使车辆保持直行的中间值。

第五章　单片机程序设计

5.1 基本原理

小车电脑通过串口通信向单片机发送指令，单片机接收到指令后，读取其中的数据，向电机和舵机发送 pwm 信号。串口通信波特率为 38400，8 位数据位 1 位停止位，无奇偶校验。小车电脑发送指令的格式为：

字节	1	2	3	4	5	6	7
内容	0xAA	电机 PWM_L	电机 PWM_H	舵机 PWM_L	舵机 PWM_H	Check	0x55

表 4.1　通讯指令格式

注：check=电机 PWM_H+电机 PWM_L+舵机 PWM_H+舵机 PWM_L

5.2 算法设计

5.2.1 串口通讯

单这一步主要就是将对应串口的状态寄存器进行修改，根据设定的波特率初始化串口即可，注意其中存在一定的端口复用问题，但由于实际单片机所需的 GPIO 端口不多，所以实际使用中不必复用；而关于中断部分的介绍留到编解码中进行；

5.2.2 PWM 输出

首先是确定使能那个计数器端口，开启对应的时钟，然后便是设置分频比例以及初始化占空比，这一部分主要是和电脑发送过来的默认指令相一致，即输出 5V，50Hz，初始占空比为 7.5%的两路信号分别用于电机和舵机；当前暂定的两路 PWM 输出为 FTM2 的 CH0~3 其中 0,1 对应于 D0，D1 用于电机的输出，而 2,3 对应于 D2，D1 用于舵机的输出；

5.2.3 编解码与基本控制

解码部分就主要在串口的中断函数中解决，实际采用的是 UART2 的中断处理函数，其中的语句也就主要是接受首字符，判定结束字符，以及计算校验位来

判断是否有效；如果有效就将接收到的信息传递到另一个数组中待处理，然后将valid 置为 1 表示有效，返回到主函数中进行处理；

而处理的过程无非就是利用位操作拼接数据，进行必要的处理如舵机占空比的范围限制，还有就是要记得返回当前的两个 PWM 的状态给电脑，其编码方式与解码过程同理即可。

第六章　结语

本次智能汽车的制作涉及了运动控制、激光雷达、SLAM 地图构建、自主规划等人工智能专业知识。小车的制作应当兼顾软件算法与硬件电路的涉及。本队制作的小车已经能够达到自主构建地图，自主路径规划与最优路径选择。目前本队已经基本完成了小车的搭建，实现了小车按照相应指令运行。本队接下来的任务包括：系统性的参数调制，附加模块的调试与应用，ROS 操作系统的综合开发。为了能够使小车高性能的运行，本队还计划通过教材、文献对智能车领域前沿技术进行调研，争取在反复的实践中找到最优解。

附录：程序源代码

单片机译码部分

```
#include "include.h"


u8 messageReceive[8];
u8 messageValid[8] = {0xaa, 0xdc, 0x05, 0xdc, 0x05, 0xc2, 0x55};
u8 index = 0;
u8 valid = 0;



u16 Min(u16 a, u16 b)
{
   return a > b? b: a;
}

u16 Max(u16 a, u16 b)
{
   return a > b? a: b;
}



void main(void)
```

```
{
    DisableInterrupts;          //关总中断
    SPLL_Init(PLL180);
    EnableInterrupts;           //开总中断

    LED_Init();                 //初始化 LED
    OLED_Init();                //初始化 OLED
    //TFT_init();               //初始化 TFT
    UART_Init(UART_2,115200);//初始化 串口
    test();
    u16 pwmMotor = 1500, pwmSteer = 1500;


    for(;;)
    {
        u8 i;
        if (valid)
        {
            for(i = 0; i < 7; ++i)
            {
                UART_PutChar(UART_2, messageValid[i]);
                messageValid[i] = messageReceive[i];
            }
```

```
            pwmMotor = messageValid[1] + (messageValid[2] << 8);
            pwmSteer = messageValid[3] + (messageValid[4] << 8);
            pwmMotor = Min(pwmMotor, 20000);
            pwmSteer = Min(pwmSteer, 2500);
            pwmSteer = Max(pwmSteer, 500);
            messageValid[1] = (u8)(pwmMotor & 0x00ff);
            messageValid[2] = (u8)((pwmMotor & 0xff00) >> 8);
            messageValid[3] = (u8)(pwmSteer & 0x00ff);
            messageValid[4] = (u8)((pwmSteer & 0xff00) >> 8);
            u16 sum = messageValid[1] + messageValid[2] + messageValid[3] +
messageValid[4];
            sum = sum & 0x00ff;
            messageValid[5] = (u8)sum;

            FTM_PWM_Duty(FTM2, FTM_CH0, pwmMotor);
            FTM_PWM_Duty(FTM2, FTM_CH2, pwmSteer);
            valid = 0;
        }

        //LED_Ctrl(LED4, RVS);
        //LED_Ctrl(LED2, RVS);

        //time_delay_ms(1000);
    }
```

```
}
```

ROS 驱动部分

```
//
// Created by Steven Zhang on 18-12-14.
// art racecar
//


//#include "../include/art_racecar_driver.h"
#include <ros/ros.h>
#include <ros/package.h>
#include <geometry_msgs/Twist.h>

#include <serial/serial.h>

serial::Serial ser;

int code(uint16_t v,uint16_t a);

void art_racecar_init(int n,char* data){
    ser.setPort(data);
    ser.setBaudrate(n);
    //serial::Timeout to = serial::Timeout::simpleTimeout(1000);
    //ser.setTimeout(to);
```

```
    ser.open();
}

void send_cmd(uint16_t v,uint16_t a){
    ser.write(code(v,a));
}

void TwistCallback(const geometry_msgs::Twist& twist)
{
    double angle;
    //ROS_INFO("x= %f", twist.linear.x);
    //ROS_INFO("z= %f", twist.angular.z);
    angle = 2500.0 - twist.angular.z * 2000.0 / 180.0;
    //ROS_INFO("angle= %d",uint16_t(angle));
    send_cmd(uint16_t(twist.linear.x),uint16_t(angle));
}

int main(int argc, char** argv)
{
    char data[] = "/dev/USBtty0";
    art_racecar_init(38400,data);
    ros::init(argc, argv, "art_driver");
    ros::NodeHandle n;
```

```
    ros::Subscriber sub = n.subscribe("/car/cmd_vel",1,TwistCallback);


    ros::spin();


}
```